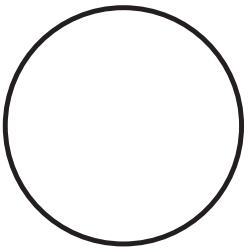
3

TOOLS AND FRAMEWORKS



This chapter will provide you with a broad understanding of the fuzzing tools available for use with the AOSP and how to work with them. You'll gain a deeper understanding of sanitizers, which allow fuzz targets to find actionable crashes. You'll learn about fuzzing engines, the core components that run the fuzzing loop, generate inputs, measure test coverage, and guide mutations. You'll explore analysis techniques for measuring the effectiveness of your fuzz harness and finding gaps. And, finally, you'll work with data-handling tools, which enable you to quickly and effectively process the data generated by the fuzzing engine.

Sanitizers

In Chapter 1, we mentioned that many of the most severe vulnerabilities don't cause an immediate or obvious crash. Instead, they can manifest as

silent memory corruptions or logical errors from undefined behavior, leading to bugs caused by one section of code that surface in a completely different area. Without sanitizers, a fuzzer might trigger such a bug thousands of times without causing any observable crashes or might generate crashes that point to the wrong code.

Sanitizers solve this problem. These powerful instrumentation tools, which are built into the compiler, add runtime checks to a program to detect hidden bugs the moment they occur. When a sanitizer finds an issue, it stops the program and generates a report to pinpoint the exact location and cause of the bug. The most popular sanitizers to use are as follows:

AddressSanitizer (ASan) The cornerstone sanitizer for fuzzing, designed to catch a wide range of memory errors such as heap and stack buffer overflows, as well as use-after-free, double-free, and other invalid memory accesses. On physical Android devices, you'll use the hardware-assisted version of ASan, called *HWASan*.

Memory Tagging Extension (MTE) A hardware feature on modern ARMv9 CPUs that provides a highly efficient alternative to ASan. It will catch the same categories of memory safety errors as ASan, but because the checks are performed in hardware, MTE has significantly lower performance overhead than ASan's software-based approach.

UndefinedBehaviorSanitizer (UBSan) Detects operations that the C and C++ language standards define as undefined behavior. These include signed integer overflow, invalid bit shifts, misaligned memory accesses, and invalid type casting.

MemorySanitizer (MSan) Can detect the use of uninitialized memory by catching reads from variables that have been allocated but not yet written to. Reading from uninitialized memory can introduce nondeterministic behavior and leak information from leftover data in memory.

Sanitizers can significantly impact the performance of your code, slowing your fuzzer's number of executions per second. ASan typically adds a performance overhead of around 100 percent, while MTE may add an overhead in the tens-of-percent range (although these percentages will vary widely). So, to test Android, sticking with just ASan or MTE is usually best, to catch the most common and critical bug classes while keeping the overhead manageable.

The difference between these two sanitizers is in the way they are implemented. ASan works by creating a region of *shadow memory*. For every 8 bytes of application memory, ASan reserves 1 shadow byte to track its state, which may be addressable, partially addressable (that is, only the first k bytes are accessible), or poisoned (meaning the program should not be reading from or writing to this location). If the program then accesses an address that is poisoned, ASan reports a violation.

Red zones are regions of memory that are completely poisoned and placed adjacent to an allocation. Through the compile-time instrumentation, ASan adds checks on memory accesses so that if code reads or writes into a poisoned red zone (for example, by overrunning an array), it reports the out-of-bounds error.

MTE works by assigning a 4-bit tag to both the pointers and each 16-byte chunk of memory. When code uses a pointer to access memory, the hardware checks that the pointer's tag matches the memory block's. If they don't match, MTE will catch this. One downside of this approach is that since the tag is only 4 bits long, there is a 1 in 16 chance that the tag from a wayward pointer will coincidentally match a memory block's tag that doesn't belong to it. This means there is a small probability that MTE will miss an error during a single execution, but because tags are randomized upon allocation, repeated executions increase the chance of observing a mismatch and catching the issue.

Fuzzing Engines

A fuzzing engine is a tool responsible for generating fuzzing inputs (together called the corpus), executing the target program, and analyzing the results. Many fuzzing campaigns select one primary engine, then may add a secondary engine to apply different strategies to the same corpus. For example, the most common fuzzer in the Android ecosystem is libFuzzer, but many testers broaden their coverage with AFL++. This section provides an overview of several popular fuzzing engines.

libFuzzer

The libFuzzer coverage-guided fuzzing engine is part of the LLVM/Clang toolchain and is linked to the fuzz harness at compile time. It runs in process, allowing a very high execution speed, and is integrated with the compiler, so creating a fuzzer is often as simple as adding a few new build flags. One limitation is that the code you fuzz must be built through Clang.

While libFuzzer is still widely used, stable, and very effective, it's in maintenance mode, and its performance has fallen slightly behind modern, actively developed fuzzing engines such as AFL++.

AFL++

AFL++ is a community-maintained fork of the legendary American Fuzzy Lop (AFL) engine. This high-performance, out-of-process, coverage-guided fuzzing engine fuzzes the target code in a separate process through the use of a *fork server*. This forks a new child from a clean snapshot of the process for each input, ensuring that the global state doesn't become corrupted and that all crashes found are reproducible.

AFL++ also has a *persistence* mode, in which the developer can specify the number of inputs to feed the function before creating a new process and clearing the global state. Global state changes will still accumulate as the engine supplies inputs for the same forked process, but this approach strikes a balance between speed and maintaining a clean global state for reproducibility.

AFL++ is great for nearly any fuzzing scenario. It supports compiler-based instrumentation for code built through Clang or the GNU Compiler Collection (GCC) and is also able to instrument binary targets with dynamic instrumentation via QEMU or Frida. The active and dedicated community surrounding AFL++ makes it one of the most effective fuzzing engines available.

LibAFL

Unlike the other tools discussed so far, LibAFL isn't a monolithic fuzzing engine but a highly modular framework for building your own custom fuzzers. It provides a rich set of reusable components, allowing you to assemble a customized fuzzer to fit your needs. With LibAFL, you can mix and match the following primitives:

Executor Runs the target in the specified way (such as in process, fork server, or QEMU)

Observer Gathers feedback about various metrics (such as code coverage, execution time, and number of system calls)

Feedback Consumes output from the observers to decide whether an input is interesting

Scheduler Sets a strategy for selecting the next input from the corpus to mutate

Mutator Applies mutation strategies (such as bit flipping, splicing two corpus items together, and so on)

Because of its high customizability, LibAFL is an excellent choice for complex targets, research, and scenarios where other fuzzing engines fall short.

Syzkaller

Syzkaller is a specialized coverage-guided fuzzing engine designed for fuzzing operating system kernels. It generates sequences of system calls to find bugs in the kernel's API. By using a high-level domain-specific language called *Syzlang*, Syzkaller defines syscalls and their complex relationships, allowing it to generate valid interactions. We cover Syzkaller in more detail in Chapter 9.

CHOOSING A FUZZING ENGINE

The simplest way to choose a fuzzing engine is to follow this rule: For standard Android components, start with libFuzzer and sanitizers because of its native integration and support in the AOSP build system. If you are targeting a project that can be built with Clang, libFuzzer is usually the easiest place to begin. If you need binary-only or mixed targets, AFL++ gives you flexible modes while still getting feedback-guided mutation. If you have moved on to very advanced fuzzing techniques or are experimenting with new approaches to fuzzing, consider LibAFL. If you are targeting the kernel, go with Syzkaller.

Data Parsing

A fuzzer has no way of distinguishing an image parser from a network protocol handler, a text-rendering library, or any other type of software under test. Therefore, the fuzzer must use the limited amount of data fed back to it to develop an understanding of which inputs were (or will be) interesting.

In practice, this understanding will come from a fuzz harness you'll write. The fuzz harness will accept a buffer of raw bytes from the fuzzing engine, then interpret those raw bytes and turn them into the typed arguments that the target function expects.

Let's discuss how to manually parse this data, then cover useful tools that will make this process less tedious and error prone. The function shown next will serve as our target. To follow along, create a new directory at *tools/security/fuzzing/parsing_examples/*, then create two new files in that directory, *test_function.cpp* and *Android.bp*. (For more on the *Android.bp* file, revisit "The Soong Build System" on page 19.) Add the following content to *test_function.cpp*:

test_function.cpp

```
#include <cstdint>
#include <string>

void myTestFunction(uint32_t magic_value, const std::string& user_name,
                    uint8_t flags, int16_t transaction_id, bool is_premium) {

    if (magic_value != 0xDEADBEEF) {
        return;
    }

    if (is_premium && !user_name.empty()) {
        if ((flags & 0x80) && transaction_id > 0) {
            __builtin_trap();
        }
    }
}
```

This vulnerable function takes structured arguments, then runs through numerous conditionals, which, if all satisfied, will trigger a crash. Now let's see how to make the inputs supplied by the fuzzer more likely to trigger this issue.

Manual Parsing

A straightforward way to turn the byte buffer that's passed in by the fuzzing engine into the typed inputs expected by our target function is to manually decode the necessary values from the raw input bytes. Here's an example fuzz harness that uses this approach to extract the values needed to call `myTestFunction()`:

manual_parsing_fuzzer.bp

```
#include <cstdint>
#include <cstddef>
#include <string>
#include "test_function.cpp" // Include the function to be tested.

extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
 ❶ if (size < sizeof(uint32_t)
              + sizeof(uint8_t)
              + sizeof(uint8_t)
              + sizeof(int16_t)
              + sizeof(bool)) {
        return 0;
    }

    size_t offset = 0;

    uint32_t magic_value = *reinterpret_cast<const uint32_t*>(data + offset);
    offset += sizeof(uint32_t);

    uint8_t name_len = *(data + offset);
    offset += sizeof(uint8_t);

    if (size < offset + name_len) {
        return 0;
    }
    std::string user_name(reinterpret_cast<const char*>(data + offset), name_len);
    offset += name_len;

 ❷ if (size < offset + sizeof(uint8_t) + sizeof(int16_t) + sizeof(bool)) {
        return 0;
    }

    uint8_t flags = *(data + offset);
 ❸ offset += sizeof(uint8_t);
```

```
    int16_t transaction_id = *reinterpret_cast<const int16_t*>(data + offset);
    offset += sizeof(int16_t);

    bool is_premium = static_cast<bool>(data[offset]);
    offset += sizeof(bool);

    myTestFunction(magic_value, user_name, flags, transaction_id, is_premium);

    return 0;
}
```

First, this function verifies that the size of the buffer is large enough to populate all the values ❶. You'll notice that, since strings can be variable lengths, we need to check the size of the remaining data after assigning the string value ❷. We also need to track our position in the buffer; to do this, we keep a variable, `offset`, and increment it after each assignment ❸.

Here is the corresponding *Android.bp* file entry:

Android.bp

```
// Fuzzer 1: Manual Parsing
cc_fuzz {
    name: "manual_parsing_fuzzer",
    srcs: [
        "manual_parsing_fuzzer.cpp",
    ],
}
```

This harness involves a lot of complexity for parsing only a few simple values from the input. In particular, the manual approach comes with two specific drawbacks:

Constant size checks Before each piece of data, we must manually check its size to ensure that we don't read past the end of the buffer. An off-by-one error here will introduce a bug into the harness itself, causing it to crash frequently before we can explore the code more deeply.

Pointer arithmetic and casting In our example, we've manually cast `char*` pointers to read various data types. This can easily introduce more bugs into the harness, leading to time wasted investigating bugs in the harness when we could be fuzzing.

You may also notice an issue with our manual parsing code. Since the `offset` variable depends on the dynamic length of the user string, it may point to an unaligned memory address when this line executes. Since we're casting to `int16_t`, which is 2 bytes in size, the pointer created by `reinterpret_cast` will be unaligned if the offset is odd and will result in undefined behavior.

If you have UBSan enabled, it will report this error, producing noisy reports, and block the fuzzer from reaching the actual target. On some architectures, this error can even trigger a hardware exception and immediately kill the process. To avoid this, you can manually realign the offset as needed. In practice, the simplest way would be to use `memcpy()`, but this illustrates the unseen complexities in parsing even a handful of simple values.

Altogether, manually parsing values from raw bytes can be tricky to get right; in fact, it's common to fuzz parsing functions because of this difficulty. Not only might there be bugs in the parsing logic causing the harness to crash, but the harness may also fail to produce a wide range of values. For example, if a harness needs an integer modulo 500, a developer might write `num = data[0] % 500`, unintentionally limiting the fuzzer to values from 0 to 255 because `data[0]` is a single byte.

Now it's time for you to become familiar with the fuzzing tools you can use to skip the headache of writing parsing logic and debugging fuzz harness crashes.

With the FuzzedDataProvider Library

Instead of constantly writing and rewriting parsing logic, you might want to go ahead and make this code a reusable library. Well, you're in luck; the fuzzing framework already provides a class called `FuzzedDataProvider`. This class wraps the raw byte stream in a much safer and more developer-friendly API. Here is the harness rewritten using `FuzzedDataProvider`:

fdp_fuzzer.cpp

```
#include <cstdint>
#include <string>
#include <fuzzer/FuzzedDataProvider.h>
#include "test_function.cpp"

extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    FuzzedDataProvider provider(data, size);

    uint32_t magic_value = provider.ConsumeIntegral<uint32_t>();
    std::string user_name = provider.ConsumeRandomLengthString();
    uint8_t flags = provider.ConsumeIntegral<uint8_t>();
    int16_t transaction_id = provider.ConsumeIntegral<int16_t>();
    bool is_premium = provider.ConsumeBool();

    myTestFunction(magic_value, user_name, flags, transaction_id, is_premium);
    return 0;
}
```

We begin by initializing the `FuzzedDataProvider` object with the raw data pointer and its size. Then we skip all the manual parsing by using it to provide all our values, keep track of the offset, and ensure that our values are parsed properly.

Here is the *Android.bp* entry for the FuzzedDataProvider fuzzer:

Android.bp

```
//
--snip--
cc_fuzz {
    name: "fuzzed_data_provider_fuzzer",
    srcs: [
        "fdp_fuzzer.cpp",
    ],
}
```

This fuzzer significantly improves the safety and readability of the parsing, as FuzzedDataProvider will handle all boundary checks internally. If you ask for more data than is available, it will gracefully return a default value (such as `0`, an empty string, or `false`), averting a crash and allowing the fuzzer to continue fuzzing. This increases the likelihood of a crash being the result of an error in the target code rather than in the harness.

Using FuzzedDataProvider is an easy way to begin cleaning up your fuzz harness code and is the right tool to use in many cases. But we can make the input generated by the fuzzing engine even more targeted.

With the libprotobuf-mutator Library

In both of the previous examples, the fuzzing engine had limited information about the data it generated. For instance, in the manual parsing fuzzer, we decided on the length of the string we created by reading an arbitrary set of bytes from the input data, but the fuzzing engine doesn't know that this part of the input will determine a string length, and it doesn't know which bytes of the input will be read as a string.

In this section, we'll look at libprotobuf-mutator, which uses structure-aware mutations to create the input, giving the fuzzing engine a formal map of the input structure. This library works by mutating instances of a Protocol Buffers (Protobufs) message, then passing that message into your harness. If you haven't worked with Protobufs before, know that they're a language-neutral way to serialize data and move it around, similar to XML or JavaScript Object Notation (JSON).

To use this library, we'll need to create a Protobuf schema in a *.proto* file to define the fields we'll use in our new fuzz harness. You'll also notice that we're no longer including the `LLVMFuzzerTestOneInput()` function. Here is the content of the *example.proto* file:

example.proto

```
syntax = "proto2";

message MyTestInput {
  optional uint32 magic_value = 1;
  optional string user_name = 2;
  optional uint32 flags = 3;
  optional int32 transaction_id = 4;
  optional bool is_premium = 5;
}
```

Protobufs don't have 8-bit or 16-bit integer types, so we use 32-bit fields and explicitly narrow them in the harness. Here is the fuzz harness, *lpm_fuzzer.cpp*, using the `DEFINE_PROTO_FUZZER` macro:

lpm_fuzzer.cpp

```
#include <cstdint>
#include <string>
#include "example.pb.h"
#include "test_function.cpp"

#include "src/libfuzzer/libfuzzer_macro.h"

DEFINE_PROTO_FUZZER(const MyTestInput& input) {
    uint32_t magic_value = input.magic_value();
    std::string user_name = input.user_name();
    uint8_t flags = static_cast<uint8_t>(input.flags());
    int16_t transaction_id = static_cast<int16_t>(input.transaction_id());
    bool is_premium = input.is_premium();

    myTestFunction(magic_value, user_name, flags, transaction_id, is_premium);
}
```

We use *example.pb.h* to include our Protobufs-generated header. Notice the input is a structured object, not raw bytes, so we need only read our values from the passed-in Protobuf, cast some of them to their expected types, and call our function. Here's the *Android.bp* file for the Protobuf mutator approach:

Android.bp

```
--snip--
cc_library_static {
    name: "example_input_proto",
    srcs: ["example.proto"],
    proto: {
        export_proto_headers: true,
    },
    shared_libs: ["libprotobuf-cpp-full"],
}

cc_fuzz {
    name: "lpm_fuzzer",
    srcs: [
        "lpm_fuzzer.cpp",
    ],
    static_libs: [
        "example_input_proto",
        "libprotobuf-mutator",
    ],
```

```
    shared_libs: [
        "libprotobuf-cpp-full",
    ],
}
```

The wonderful thing about using this structure-aware approach is it requires no manual parsing. The `DEFINE_PROTO_FUZZER` macro and `libprotobuf` mutator engine handle the complex and error-prone task of translating raw bytes into a fully populated `MyTestInput` object. In addition, the fuzzing engine is smart; it understands that `magic_value` is a 4-byte integer and can set this field to a known interesting value, such as `0`, `-1`, or `MAX_INT`. We've also just made this fuzzer easy to maintain. If the function's signature changes in the future, we can simply update the Protobuf in the *.proto* file to match, pass in the new field, and begin fuzzing.

Automating the Fuzzing Life Cycle

While the tools discussed so far are excellent for creating and running fuzzers locally, in the long term you'll want an automated solution that can continuously build fuzz targets and hunt for vulnerabilities. Open source infrastructure called ClusterFuzz can meet this need. Originally developed by Google, it serves as the backbone for fuzzing massive projects like Google Chrome and AOSP, providing automation at a scale that would be impossible to manage manually.

ClusterFuzz automates the entire fuzzing life cycle. A developer submits a fuzz target, and the platform handles the rest: building the code with various sanitizers, running the fuzzer 24/7 on a massive fleet of computers, and processing the results. After finding a crash, ClusterFuzz automatically de-duplicates it, minimizing the test case to its smallest form, and files a detailed bug report. ClusterFuzz can even identify the exact commit that introduced the vulnerability, freeing developers from manual triage so they can focus on the fix.

Corpus minimization is a process that removes new corpus entries if they don't add more coverage. As fuzzers run, they discover new random inputs that provide more coverage, and we hope that if errors exist or are added to the code that is covered, we'll find the issue with fuzzers. However, over time, as code changes, or depending on other parameters, you will inevitably end up having more and more fuzzer inputs—too many fuzzer inputs. If these inputs produce the same coverage, running these duplicated inputs won't provide any more value. So, if you run a fuzzer for any amount of time, you typically want to minimize the coverage, at least sometimes.

While you can set up your own instance of ClusterFuzz, doing so takes significant effort. For most open source projects, it's much easier to follow ClusterFuzz's online documentation to onboard to Google's instance, OSS-Fuzz, which is free for open source projects. OSS-Fuzz currently supports more than 1,100 projects. You can find its onboarding documentation at *https://google.github.io/oss-fuzz/getting-started/new-project-guide/*.

Analyzing Lines Covered

To understand which parts of the code your fuzzer is testing, you can generate a coverage report, which visually shows which lines were executed and which were missed. This section will guide you through the process of generating a detailed report, then strengthening your fuzzer based on the results from that report.

NOTE *At the time of this writing, generating coverage reports works on physical devices only.*

You generate a coverage report in three main steps: building the fuzz target with coverage instrumentation, running the instrumented fuzzer to generate the raw profile data, and processing the data on your local machine to generate the report. To build the fuzz target, navigate to the AOSP root. Use the same terminal window throughout this process, as you'll be setting new shell variables throughout.

These commands will build the target with coverage instrumentation:

```
$ source build/envsetup.sh
$ lunch aosp_arm64-trunk_staging-userdebug
$ CLANG_COVERAGE=true NATIVE_COVERAGE_PATHS=* m libskia_image_processor_fuzzer -j8
```

Next, push the output of your build to your device and run the fuzzer for 5,000 executions to generate profile data:

```
$ cd "$ANDROID_PRODUCT_OUT/data/fuzz/$(get_build_var TARGET_ARCH)"
$ adb push * /data/local/tmp
$ adb shell
$ cd data/local/tmp/libskia_image_processor_fuzzer
$ LLVM_PROFILE_FILE=profdata/out.profraw ./libskia_image_processor_fuzzer -runs=5000
```

You should see results like the following after running your fuzzer:

```
#963    NEW    cov: 361 ft: 397 corp: 26/88b lim: 4 exec/s: 0 rss: 45Mb L: 4/4 MS: 2 Cha-
#1050   NEW    cov: 361 ft: 399 corp: 27/92b lim: 4 exec/s: 0 rss: 45Mb L: 4/4 MS: 2 Ins-
#1156   NEW    cov: 361 ft: 400 corp: 28/96b lim: 4 exec/s: 0 rss: 45Mb L: 4/4 MS: 1 Cr-
#1242   NEW    cov: 361 ft: 401 corp: 29/100b lim: 4 exec/s: 0 rss: 45Mb L: 4/4 MS: 1 Ch-
#1303   NEW    cov: 361 ft: 402 corp: 30/104b lim: 4 exec/s: 0 rss: 45Mb L: 4/4 MS: 1 Ch-
#1321   NEW    cov: 361 ft: 408 corp: 31/108b lim: 4 exec/s: 0 rss: 45Mb L: 4/4 MS: 3 Cr-
#1392   NEW    cov: 361 ft: 409 corp: 32/112b lim: 4 exec/s: 0 rss: 45Mb L: 4/4 MS: 1 Ch-
#1432   NEW    cov: 361 ft: 410 corp: 33/116b lim: 4 exec/s: 0 rss: 45Mb L: 4/4 MS: 5 Ch-
```

```
#1504   NEW    cov: 361 ft: 411 corp: 34/120b lim: 4 exec/s: 0 rss: 45Mb L: 4/4 MS: 2 Ch-
#1620   NEW    cov: 361 ft: 412 corp: 35/124b lim: 4 exec/s: 0 rss: 45Mb L: 4/4 MS: 1 Cr-
#1781   NEW    cov: 361 ft: 413 corp: 36/128b lim: 4 exec/s: 0 rss: 45Mb L: 4/4 MS: 1 Ch-
#1987   NEW    cov: 361 ft: 414 corp: 37/134b lim: 6 exec/s: 0 rss: 46Mb L: 6/6 MS: 1 In-
        NEW_FUNC[2/2]: 0x55dc11b3f340 in SkSwizzler::swizzle(void*, unsigned char const*-
#2265   NEW    cov: 367 ft: 421 corp: 39/146b lim: 8 exec/s: 0 rss: 51Mb L: 6/6 MS: 1 Co-
#2266   NEW    cov: 367 ft: 422 corp: 40/154b lim: 8 exec/s: 0 rss: 51Mb L: 8/8 MS: 1 Pe-
#2310   NEW    cov: 368 ft: 423 corp: 41/161b lim: 8 exec/s: 0 rss: 51Mb L: 7/8 MS: 4 Ch-
#2381   NEW    cov: 368 ft: 426 corp: 42/165b lim: 8 exec/s: 0 rss: 51Mb L: 4/8 MS: 1 Ch-
#2400   NEW    cov: 370 ft: 431 corp: 43/173b lim: 8 exec/s: 0 rss: 51Mb L: 8/8 MS: 4 Pe-
#2461   NEW    cov: 370 ft: 432 corp: 44/181b lim: 8 exec/s: 0 rss: 51Mb L: 8/8 MS: 1 Ch-
#2472   NEW    cov: 370 ft: 438 corp: 45/185b lim: 8 exec/s: 0 rss: 51Mb L: 4/8 MS: 1 Ch-
```

You should also now have a directory alongside your fuzz binary called *profdata*. Inside *profdata*, you'll have a file called *out.profraw* that contains the raw data, tracking which lines of code were executed by the fuzzer.

Because these coverage tools in AOSP can sometimes break, let's make sure everything works as expected before continuing by printing the data in *out.profraw* to the screen:

```
$ ls -lh profdata/out.profraw
-rw-rw-rw- 1 shell shell 31M 12-01 18:54 profdata/out.profraw
```

You can see this file is 31MB in size; this means the coverage data was collected. Otherwise, the file size would show 0. Coverage sometimes breaks in AOSP, so if the data isn't being collected, you should still follow along with this book to understand the process.

The next step is to pull this data off the device so you can work with it on your local machine. Here are the commands to retrieve the profile data:

```
$ exit
$ cd $ANDROID_BUILD_TOP
$ adb pull /data/local/tmp/libskia_image_processor_fuzzer/profdata/
```

You'll use `llvm-profdata` to merge and format the data and `llvm-cov` to take that data and generate the reports. These tools ship alongside Clang as part of the LLVM project. To ensure that you have no versioning issues, use the tools in AOSP by updating the PATH variable:

```
$ export PATH=$ANDROID_BUILD_TOP/prebuilts/clang/\
host/linux-x86/llvm-binutils-stable:$PATH
```

Now you can generate the report. First, merge and format the data via `llvm-profdata`:

```
$ llvm-profdata merge --sparse profdata/* --output coverage.profdata
```

To create the report, it's critical that you use the unstripped binary containing the debug information, located in the symbols directory of the Soong

output. These commands define the symbols path and generate the initial report:

```
$ FUZZ_SYMBOLS_DIR=$ANDROID_PRODUCT_OUT/symbols/\
data/fuzz/$(get_build_var TARGET_ARCH)
$ SYMBOLS_PATH=$FUZZ_SYMBOLS_DIR/libskia_image_processor_fuzzer/\
libskia_image_processor_fuzzer
$ llvm-cov show --format=html --instr-profile=coverage.profdata \
--path-equivalence=/proc/self/cwd/,$ANDROID_BUILD_TOP $SYMBOLS_PATH \
--output-dir=coverage_report
```

Open *index.html* from the *coverage_report* directory to see your report. You should see an overwhelming number of files in this report, many of which concern libraries with no coverage, and others that relate to low-level shared libraries not interesting for finding vulnerabilities in the image-processing code, such as logging. To remove these files and make the report easier to work with, use the flag `--ignore-filename-regex`, then generate a new report:

```
$ IGNORE_PATHS='
--ignore-filename-regex=.*/prebuilts/.*
--ignore-filename-regex=.*/bionic/.*
--ignore-filename-regex=.*/external/libyuv/.*
--ignore-filename-regex=.*/external/perfetto/.*
--ignore-filename-regex=.*/external/rust/.*
--ignore-filename-regex=.*/system/logging/.*
--ignore-filename-regex=.*/usr/include/.*'
$ llvm-cov show --format=html --instr-profile=coverage.profdata \
--path-equivalence=/proc/self/cwd/,$ANDROID_BUILD_TOP $SYMBOLS_PATH \
--output-dir=coverage_report_with_ignores $IGNORE_PATHS
```

You'll find your new coverage report at *coverage_report_with_ignores/index.html*. Once you've created a base set of paths to ignore, you can reuse them for most of your future fuzz targets, adjusting the list slightly to suit each new target's needs.

Now that you've removed the content you don't want to see, add in more of what you do want to see. Currently, the reports show only the code built into the fuzz binary, comprising the fuzz harness and statically linked libraries. To see shared libraries, you'll need to use the `-object` flag for each library you want to include.

View the list of all the available libraries with this command:

```
$ ls $ANDROID_PRODUCT_OUT/symbols/data/fuzz/$(get_build_var TARGET_ARCH)/lib/
```

Since we're running the test with an image-processing fuzzer, it might be nice to look at the PNG processing library by including *libpng.so*. To do so, use this updated `llvm-cov` command:

```
$ llvm-cov show --format=html --instr-profile=coverage.profdata \
--path-equivalence=/proc/self/cwd/,$ANDROID_BUILD_TOP $SYMBOLS_PATH \
--output-dir=coverage_report_with_objects \
```

```
-object $ANDROID_PRODUCT_OUT/symbols/data/fuzz/$(get_build_var TARGET_ARCH)/lib/libpng.so \
$IGNORE_PATHS
```

If you open the new coverage report, you should see the library added, as in Figure 3-1.

Filename	Function Coverage	Line Coverage
proc/self/cwd/external/libpng/arm/arm_init.c	0.00% (0/1)	0.00% (0/18)
proc/self/cwd/external/libpng/arm/filter_neon_intrinsics.c	0.00% (0/8)	0.00% (0/249)
proc/self/cwd/external/libpng/arm/palette_neon_intrinsics.c	0.00% (0/3)	0.00% (0/85)
proc/self/cwd/external/libpng/png.c	1.49% (1/67)	0.53% (9/1688)
proc/self/cwd/external/libpng/pngerror.c	0.00% (0/19)	0.00% (0/309)
proc/self/cwd/external/libpng/pngget.c	0.00% (0/67)	0.00% (0/672)
proc/self/cwd/external/libpng/pngmem.c	0.00% (0/13)	0.00% (0/111)
proc/self/cwd/external/libpng/pngpread.c	0.00% (0/19)	0.00% (0/602)
proc/self/cwd/external/libpng/pngread.c	0.00% (0/40)	0.00% (0/2290)
proc/self/cwd/external/libpng/pngrio.c	0.00% (0/3)	0.00% (0/36)
proc/self/cwd/external/libpng/pngrtran.c	0.00% (0/48)	0.00% (0/3413)
proc/self/cwd/external/libpng/pngrutil.c	0.00% (0/56)	0.00% (0/2556)
proc/self/cwd/external/libpng/pngset.c	0.00% (0/43)	0.00% (0/1016)

Figure 3-1: A report showing coverage of libpng *files*

You can now see files from the libpng library. If you look closely, however, you'll see that the libpng files all list zero lines as being covered, except one file, which has nine covered lines. This is because we're forcing our fuzzer to find a valid PNG file to pass through the fuzz target by generating 1s and 0s from scratch. With more than 5,000 executions, our fuzzer would eventually discover the PNG format as well as the other image formats, but to make your fuzzing campaigns as effective as possible, you can give the fuzzing engine a valid PNG file to show it that such files are valuable to the program it's fuzzing.

If you look at the corpus included with this fuzzer, you'll see that the author has already included a sample PNG file. Let's generate new profile data, and this time supply the fuzzer with this example file. To avoid any issues that may arise from the other corpus seeds, we'll copy this file to our own corpus directory and feed that to our fuzzer:

```
$ adb shell
$ cd /data/local/tmp/libskia_image_processor_fuzzer
$ mkdir min_corpus
$ cp corpus/test6.png min_corpus/test6.png
$ LLVM_PROFILE_FILE=profdata/out.profraw ./libskia_image_processor_fuzzer \
 -runs=5000 min_corpus
$ exit
$ cd $ANDROID_BUILD_TOP
$ FUZZ_TARGET=libskia_image_processor_fuzzer
$ FUZZ_SYMBOLS_DIR=$ANDROID_PRODUCT_OUT/symbols/\
data/fuzz/$(get_build_var TARGET_ARCH)
$ SYMBOLS_PATH=$FUZZ_SYMBOLS_DIR/$FUZZ_TARGET/$FUZZ_TARGET
$ adb pull /data/local/tmp/$FUZZ_TARGET/profdata/ .
$ llvm-profdata merge --sparse profdata/* --output coverage.profdata
```

```
$ llvm-cov show --format=html --instr-profile=coverage.profdata \
 --path-equivalence=/proc/self/cwd/,$ANDROID_BUILD_TOP $SYMBOLS_PATH \
 --output-dir=coverage_report_with_objects \
 -object $FUZZ_SYMBOLS_DIR/lib/libpng.so \
$IGNORE_PATHS
```

Figure 3-2 shows the results.

Filename	Function Coverage	Line Coverage
proc/self/cwd/external/libpng/arm/arm_init.c	100.00% (1/1)	61.11% (11/18)
proc/self/cwd/external/libpng/arm/filter_neon_intrinsics.c	62.50% (5/8)	65.46% (163/249)
proc/self/cwd/external/libpng/arm/palette_neon_intrinsics.c	0.00% (0/3)	0.00% (0/85)
proc/self/cwd/external/libpng/png.c	23.88% (16/67)	15.76% (266/1688)
proc/self/cwd/external/libpng/pngerror.c	36.84% (7/19)	29.45% (91/309)
proc/self/cwd/external/libpng/pngget.c	10.45% (7/67)	9.08% (61/672)
proc/self/cwd/external/libpng/pngmem.c	69.23% (9/13)	65.77% (73/111)
proc/self/cwd/external/libpng/pngpread.c	84.21% (16/19)	51.50% (310/602)
proc/self/cwd/external/libpng/pngread.c	12.50% (5/40)	3.97% (91/2290)
proc/self/cwd/external/libpng/pngrio.c	66.67% (2/3)	52.78% (19/36)
proc/self/cwd/external/libpng/pngrtran.c	10.42% (5/48)	4.75% (162/3413)
proc/self/cwd/external/libpng/pngrutil.c	37.50% (21/56)	17.61% (450/2556)
proc/self/cwd/external/libpng/pngset.c	11.63% (5/43)	12.01% (122/1016)

Figure 3-2: Coverage of libpng *with the PNG seed*

Compare the Line Coverage column with the one from the previous run. You should notice that, still with only 5,000 executions, most of the files now have coverage, meaning the fuzzer was able to explore this library much more effectively. By using the seed corpus, we drastically increased the effectiveness of our PNG library fuzzing.

Wrapping Up

This chapter provided a comprehensive overview of the essential tools and frameworks used in fuzzing. We covered how sanitizers like ASan and MTE are used to turn subtle memory errors into actionable crashes, explored the functionality of core fuzzing engines like libFuzzer and AFL++, and discussed advanced data-handling techniques using FuzzedDataProvider and libprotobuf-mutator. Finally, we walked through the critical process of generating and analyzing code coverage reports, demonstrating how to use seeds to significantly boost a fuzzer's effectiveness in exploring target libraries like libpng.